



{ПРОГРАММИРОВАНИЕ}

ПАТТЕРНЫ

ПРОЕКТИРОВАНИЯ

ПОРОЖДАЮЩИЕ
ПАТТЕРНЫ
ПРОЕКТИРОВАНИЯ

СТРУКТУРНЫЕ
ПАТТЕРНЫ

ПАТТЕРНЫ
ПОВЕДЕНИЯ

Урок №2

Структурные паттерны

Содержание

1. Понятие структурного паттерна	5
2. Паттерн Adapter	7
Цель паттерна	7
Причина возникновения паттерна	7
Структура паттерна	8
Результаты использования паттерна	11
Практический пример использования паттерна ..	12
3. Паттерн Bridge	15
Цель паттерна	15
Причины возникновения паттерна	15
Структура паттерна	16
Результаты использования паттерна	17
Практический пример использования паттерна ..	18

4. Паттерн Composite	20
Цель паттерна	20
Причины возникновения паттерна	20
Структура паттерна	21
Результаты использования паттерна	22
Практический пример использования паттерна	23
5. Паттерн Decorator	26
Цель паттерна	26
Причины возникновения паттерна	26
Структура паттерна	27
Результаты использования паттерна	29
Практический пример использования паттерна	30
6. Паттерн Facade	34
Цель паттерна	35
Причины возникновения паттерна	35
Структура паттерна	36
Результаты использования паттерна	38
Практический пример	38
7. Паттерн Flyweight	40
Цель паттерна	40
Причины возникновения паттерна	40
Структура паттерна	41
Результаты использования паттерна	43
Практический пример	44
8. Паттерн Proxy	46

Цель паттерна	46
Причины возникновения паттерна.	47
Структура паттерна.	48
Результаты использования паттерна	49
Практический пример	50
9. Анализ и сравнение структурных паттернов	52
10. Домашнее задание	55

1. Понятие структурного паттерна

Исходя из определения информационных систем, является очевидным, что всякое программное решение, так ли иначе, оперирует некоторой совокупностью данных, при анализе которых мы получаем некоторую информацию, позволяющую нам осуществлять принятие бизнес решений и штатных решений по управлению самой информационной системой.

Анализ данных предполагает, что они должны быть организованы в некоторой структуре, которая позволит с одной стороны идентифицировать различные данные (логически отличать их друг от друга), а с другой — организовывать атомарные величины в структурированные объекты и их совокупности. Структурирование данных необходимо для отражения понятий, которыми мы оперируем при анализе, а также отображения отношений, которые образуются между объектами в процессе их появления.

Объектно-ориентированный подход предоставляет нам богатый инструментарий в смысле описания структурированных понятий и отражения отношений, в которых они состоят. Однако организация структуры, связывающей понятия в работающую систему, имеет решающее значение с точки зрения эффективного применения информационной системы, а так же с точки зрения сопровождения полученного в результате процесса разработки продукта.

Паттерны проектирования в общем смысле отражают наилучшие модели решения прикладных проблем, прошедшие проверку временем и практикой применения. Структурные паттерны предлагают модели организации данных в структуры, которые наилучшим образом решают вопрос организации управления данными, которые мы используем в своих прикладных решениях. Следует понимать, что данными можно считать не только атомарные величины, но и все объекты, существующие в пределах нашего приложения.

Далее последует перечисление основных структурных паттернов, признанных мировым сообществом лучшей практикой разработки программного обеспечения.

2. Паттерн Adapter

Цель паттерна

Для лучшего понимания предлагаемого материала, а так же с целью упрощения изложения, нами будет введена следующая терминология, специфичная рассматриваемому вопросу: под **клиентом** (*client*) мы будем понимать некоторый класс, который использует (в общем случае агрегирует) некоторый класс, который мы называем **адаптируемым** (*adaptee*). Под **адаптером** (*adapter*) мы будем понимать класс, выполняющий приведение интерфейса адаптируемого класса к интерфейсу, ожидаемому клиентом.

Цель паттерна проектирования **Adapter** (англ. «адаптер») состоит в том, чтобы привести (адаптировать) интерфейс некоторого адаптируемого класса к интерфейсу, который ожидается клиентом.

Причина возникновения паттерна

Достаточно часто встречается следующая проблема: у нас в наборе и инструментов имеется некоторый класс, который мы хотим использовать в неспецифичной для его структуры задаче. Например, у нас объявлен тип данных, описывающий понятие сетевого устройства и названный нами `IPEndPoint`, и наделённый такими свойствами как IP-адрес, мак-адрес и имя хоста, которые мы используем в целях некоторого прикладного анализа (например, трассировки перемещения пакетов). Анализ выполняется

некоторым классом, который агрегирует множество объектов типа `IPEndPoint` и называется `NetView`. Однако мы хотим реализовать графическое представление для процесса и результата анализа, с выводом его на основное окно нашего приложения. Проблема состоит в том, что класс `NetView` не имеет интерфейса, специфичного для объекта графической подсистемы и, соответственно, не может быть использован оконным классом для выполнения прорисовки. Мы не имеем возможности «переписать» (изменить исходный текст и соответственно структуру) класс `NetView`, под нужды приложения, поскольку он является частью, используемого нами набора типов из `dll`-библиотеки предоставленной сторонними разработчиками; или мы просто не хотим «перегружать» структуру класса, поскольку она критична для каких-либо задач нашего приложения.

В таком случае логично использовать некоторый класс-посредник, который, используя наследование, позволит привести класс `NetView` к необходимому типу данных, а так же посредством переопределения методов, специфичных для компонента графической подсистемы, определит интерфейс прорисовки типа данных `NetView`.

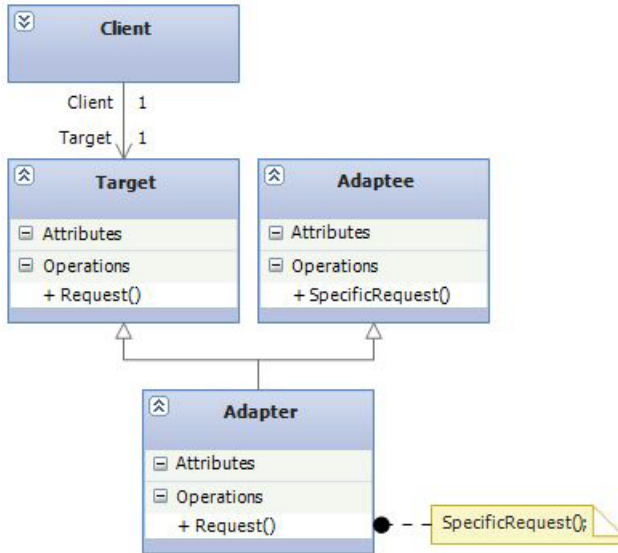
Такой класс-посредник обычно называют адаптером.

Структура паттерна

Структура паттерна `Adapter` представлена на приведённой диаграмме (см. ниже).

Участвующие элементы:

- **Client** — класс, который использует некоторые вспомогательные типы данных и ожидает, что они имеют

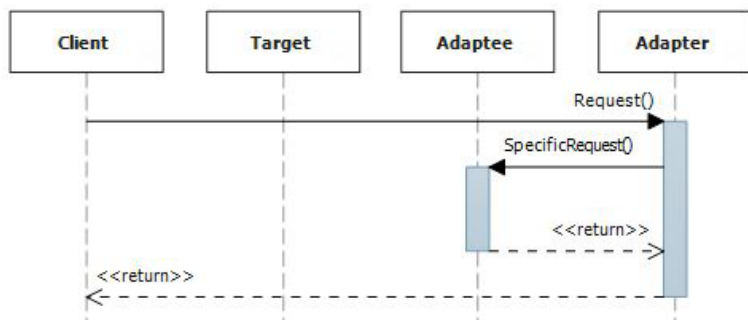


стандартный интерфейс взаимодействия (использования) описанный классом Target.

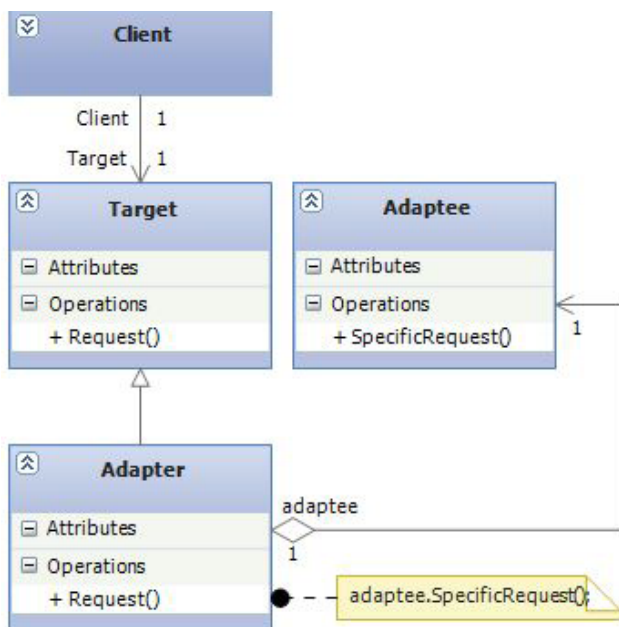
- **Target** — класс, имеющий интерфейс, ожидаемый клиентом.
- **Adaptee** — класс, который необходим для работы клиента, но имеет интерфейс, отличный от того, который ожидается клиентом.
- **Adapter** — класс, выполняющий приведение интерфейса класса Adaptee, к интерфейсу класса Target.

В, предложенной выше структуре, приведение интерфейса выполняется за счёт того, что класс Adapter наследует оба класса Adaptee и Target, а, значит, обладает интерфейсами обоих этих классов. Затем класс Adapter приводит вызовы методов специфичных для интерфейса класса Target к вызовам соответствующих методов интерфейса класса Adaptee.

На представленной ниже диаграмме последовательности демонстрируется, что вызовы методов объекта класса Adapter сводятся к вызовам методов объекта его базового класса.

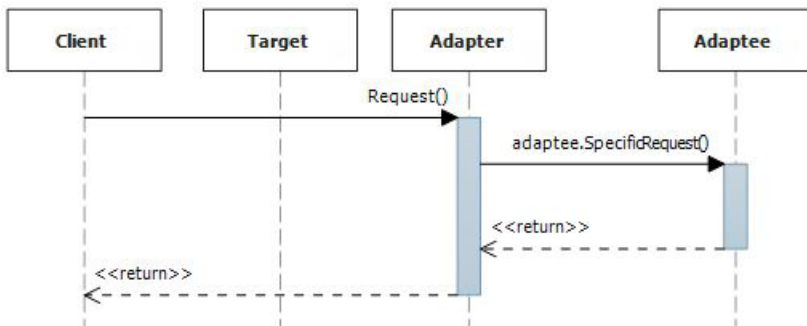


Паттерн Adapter также может быть реализован альтернативным способом, структура которого представлена на приведённой ниже диаграмме.



Отличие от предыдущей структуры состоит в том, что в текущей модели классы Adapter и Adaptee находятся не в отношении родства, а в отношении ассоциации, то есть класс Adapter агрегирует класс Adaptee.

Таким образом, приведение интерфейса класса Adaptee к интерфейсу класса Target выполняется за счёт того, что вызовы методов объекта класса Adapter, специфичные для интерфейса класса Target приводятся к вызовам соответствующий методов объекта класса Adaptee, инкапсулированного в классе Adapter. Нижеприведённая диаграмма последовательности иллюстрирует происходящее.



Результаты использования паттерна

Основной положительный результат использования паттерна проектирования Adapter состоит в том, что мы получаем возможность гибко привести интерфейс некоторого класса к интерфейсу, ожидаемому приложением без изменения структуры самого класса. Это необходимо, как с точки зрения устранения избыточности структуры типов, так и с точки зрения модульности создаваемых приложений.

Избыточность играет отрицательную роль тогда, когда нам необходимо повторно использовать написанный нами

код (например, в другом приложении) Такой код называется reusable-кодом. Создание reusable-кода считается хорошей практикой, поскольку уменьшает стоимость и увеличивает скорость разработки. А так же увеличивает гибкость и масштабируемость создаваемых приложений за счёт модульной структуры готового приложения.

Расширять приложение путём добавления новых типов значительно проще, чем полностью заново создавать некоторые модули.

Практический пример использования паттерна

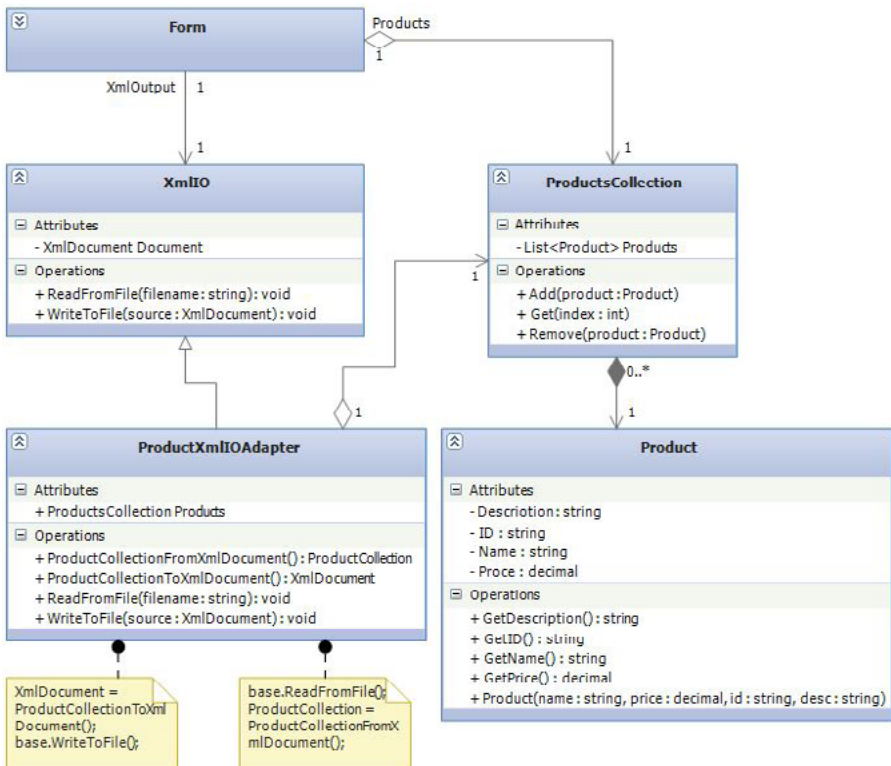
Мы рассмотрим использование паттерна Adapter на примере приложения, выполняющего управление товарооборотом некоторого предприятия. Для осуществления управления товарами мы опишем тип данных Product (продукт/товар), который будет организован в коллекцию товаров при помощи класса ProductsCollection.

Наше приложение так же выполняет управление и другими аспектами работы торгового предприятия, которые мы «опустим» для прозрачности примера.

Для реализации возможности создавать переносимую резервную копию данных, которую можно использовать для практически любых задач мы решили сделать так, чтобы можно было все данные экспортировать в отдельные xml-файлы. Для этого мы реализовали класс XmlIO, которые осуществляет запись и чтение Xml-документа, и объект которого инкапсулируется оконным классом нашего приложения. Для реализации записи в файл коллекции продуктов нами был создан класс ProductXmlIO-Adapter, который инкапсулирует коллекцию продуктов,

реализует приведение этой коллекции к xml-документу, а так же xml-документа к коллекции.

Таким образом, мы реализуем функцию записи коллекции элементов в файл в xml-формате и в то же время не изменяем структуры исходного типа данных и устраняем избыточность структуры, которая могла появиться вследствие наполнения класса Product дополнительным функционалом. Устранение избыточности необходимо постольку, поскольку тип данных Product может использоваться при приведении информации о продуктах, полученной из некоторой базы данных, к объектному представлению, для выполнения последующего анализа и так



далее. При выполнении всех этих действий избыточность структуры типа может создавать дополнительные сложности. Также, подобная модульная структура добавляет гибкости при сопровождении проекта и использовании (создании) reusable-кода.

Выше представлена диаграмма классов иллюстрирующая описанное выше приложение.

К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна Adapter.

3. Паттерн Bridge

Цель паттерна

Цель паттерна Bridge (*англ.* «мост») состоит в том, чтобы отделить абстракцию от её реализации, для того, чтобы они могли изменяться независимо друг от друга.

Причины возникновения паттерна

Обычно, в случаях, когда некоторая абстракция (обычно абстрактный класс) может иметь несколько конкретных реализаций, используют наследование для определения множества классов, с похожим (в общем случае говорят одинаковым или совместимым) интерфейсом. Абстрактный класс определяет интерфейс для своих потомков, который они реализуют «различными» способами.

Однако такой подход является не всегда достаточно гибким и имеет некоторые слабые стороны, способные привести к избыточности кода, а также создать дополнительные трудности при сопровождении проекта, что значительно увеличит его стоимость. Прямое наследование интерфейса **абстракции** некоторым конкретным классом связывает **реализацию с абстракцией** напрямую, что создаёт трудности при дальнейшей модификации **реализации** (её расширении), а так же не позволяет повторно использовать **абстракцию** и её **реализацию** отдельно друг от друга. **Реализация**, как бы, становится «жёстко связанной» с **абстракцией**.

Паттерн проектирования мост предполагает помещение интерфейса и его реализации в различных иерархиях, что позволяет отделить интерфейс от реализации и использовать их независимо, а так же комбинировать любые варианты *реализации* с различными уточнёнными вариантами *абстракции*.

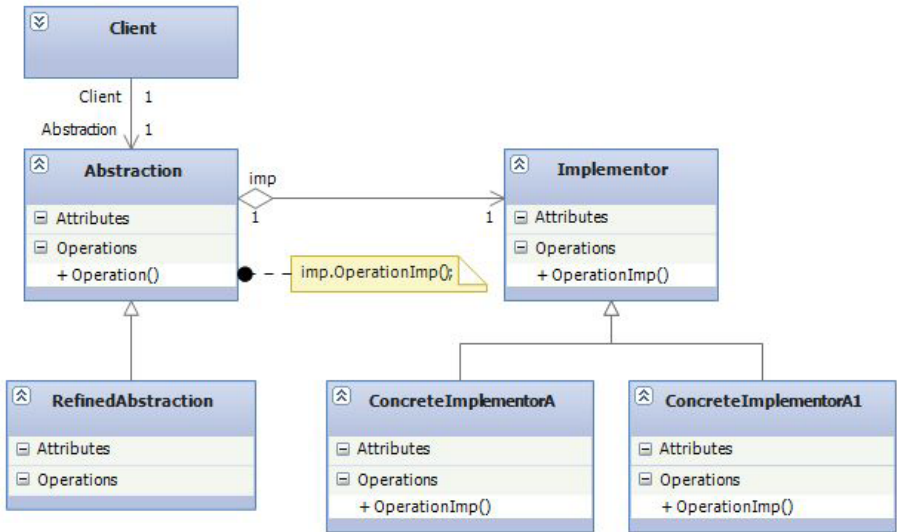
Структура паттерна

Паттерн проектирования мост представлен следующими структурными элементами:

- ***Abstraction*** (*абстракция*) — определяет интерфейс абстракции, а также содержит объект исполнителя, который определяет интерфейс реализации.
- ***Implementor*** (*исполнитель*) — определяет интерфейс для классов реализации. Интерфейс исполнителя не обязательно должен соответствовать интерфейсу абстракции. В принципе, интерфейсы, определённые абстракцией и исполнителем, могут быть совершенно разными, что является достаточно гибким. В целом, исполнитель должен определять базовые операции, на которых впоследствии базируется высокоуровневая логика абстракции.
- ***RefinedAbstraction*** (*уточнённая абстракция*) — расширяет интерфейс определённый абстракцией.
- ***ConcreteImplementor*** (*конкретизированный исполнитель*) — класс, который реализует интерфейс исполнителя и определяет его частную реализацию.

Абстракция и исполнитель совместно образуют «мост», который связывает уточнённую абстракцию с конкретной реализацией.

Структура паттерна проектирования мост представлена ниже в виде диаграммы классов.



Результаты использования паттерна

Основное преимущество которое предоставляет использование паттерна проектирования мост состоит в том, что выполняется логическое и структурное разделение абстракции от её реализации, что делает код более гибким. Так же применение паттерна мост улучшает такое качество кода, как расширяемость, поскольку абстракция и исполнитель находятся в различных иерархических структурах, а значит становиться возможным расширение реализации независимо от абстракции, и наоборот.

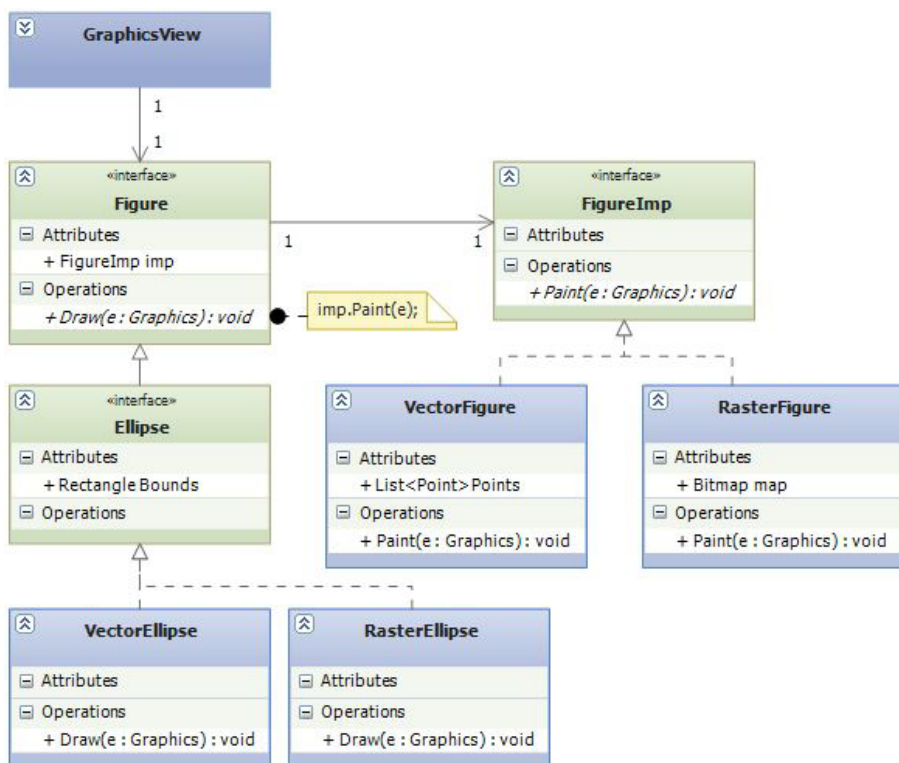
Ещё одной причиной в пользу применения паттерна проектирования мост служит тот факт, что он позволяет скрывать детали реализации от клиента (*client*), то есть от приложения, которое использует абстракцию, что

позволяет клиенту быть независимым от того, какая именно реализация была выбрана в том или ином случае.

Практический пример использования паттерна

Мы рассмотрим применение паттерна проектирования мост на примере «смешанного» графического редактора (редактора, позволяющего совместно, в рамках одного представления, редактировать растровую и векторную графику).

Модель приложения предполагает наличие некоторого графического представления, оперирующего некоторыми абстрактными фигурами, интерфейс взаимодействия



с которыми описывается интерфейсом `Figure`, играющим роль абстракции в данном примере.

Интерфейс исполнителя определяется интерфейсом `FigureImp`, от которого мы наследуем классы `VectorFigure` и `RasterFigure` — соответственно, описывающих реализации прорисовки векторной и растровой фигур.

После определения реализации мы можем уточнить абстракцию, описав интерфейса некоторой конкретной фигуры (например, эллипса). После этого можно связать уточнённую абстракцию с конкретной реализацией, например, определив классы `VectorEllipse` и `RasterEllipse`, описывающие соответственно векторный и растровый эллипсы.

На рисунке выше представлена модель описанного приложения.

К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна Bridge.

4. Паттерн Composite

Цель паттерна

Паттерн Composite (компоновщик) предназначен для того, чтобы представить объекты в виде структуры дерева в иерархической связи часть-целое.

Причины возникновения паттерна

Паттерн компоновщик существует потому, что структура типа «дерево» является достаточно распространённой и часто используется для организации данных имеющих регулярную структуру. То есть такую структуру, которая реализует иерархическую зависимость часть-целое, предполагающую, что элементом некоторой композиции данных может быть не только элементарный элемент, но и такая же композиция.

Самая простая реализация подобной структуры с рекурсивной системой вложенности может быть выражена в виде некоторой системы типов, содержащей классы, описывающие элементарные компоненты, и классов, которые будут использоваться в качестве контейнеров для элементарных компонент.

Но подобный подход имеет серьёзный недостаток, состоящий в том, что код, использующий указанные классы, должен отдельно обрабатывать элементы и их контейнеры, даже если в большинстве случаев они обрабатываются одинаково. Это резко усложняет реализацию приложения.

Паттерн компоновщик предлагает рекурсивную структуру, при которой не придётся принимать решения о том, как обрабатывать отдельные элементы общей совокупности данных.

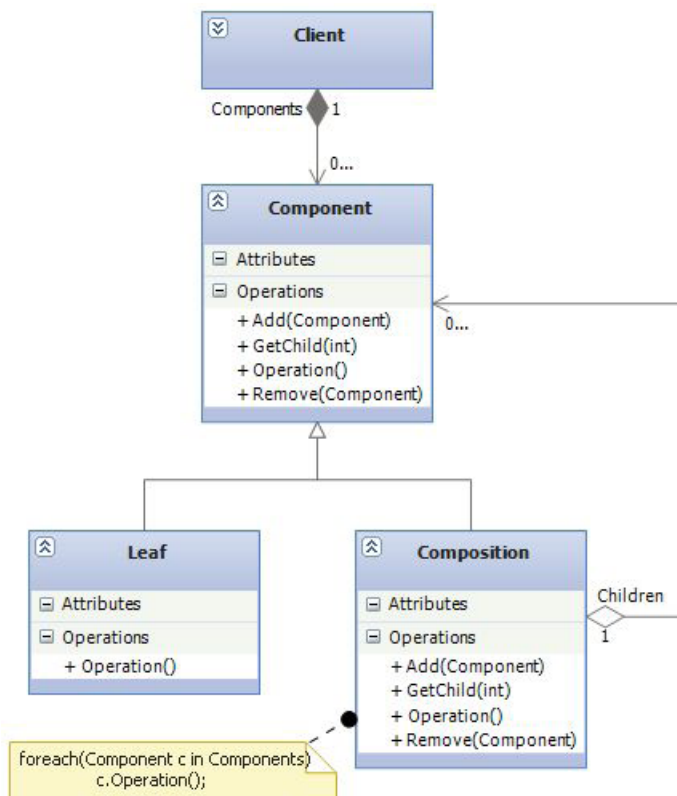
Ключ к пониманию природы компоновщика состоит в определении абстрактного идентичного интерфейса для элементарных компонент и их контейнеров. Таким образом, поскольку контейнеры и элементарные компоненты находятся в отношении родства и имеют общий пользовательский интерфейс, то контейнер может быть элементом другого такого же контейнера, что создаёт удобную регулярную (рекурсивную) структуру.

Структура паттерна

- ***Component*** (компонент) — описывает интерфейс для объектов и их композиций; реализует базовое поведение, специфичное и для отдельных элементов и для их композиций; определяет интерфейс доступа к элементам композиции и управления этими элементами, а также определяет интерфейс доступа к родительскому элементу рекурсивной структуры.
- ***Leaf*** (лист) — определяет отдельный элемент композиции и описывает поведение «примитивных» (базовых) элементов общей структуры.
- ***Composition*** (композиция) — определяет поведение для компонентов, содержащих дочерние элементы, инкапсулирует дочерние элементы, а также реализует операции управления дочерними элементами и доступа к ним, определённые интерфейсом компонента.

- **Client** (клиент) — управляет элементами композиции через интерфейс компонента.

Структура паттерна компоновщик представлена в виде диаграммы классов, приведённой на рисунке.



Результаты использования паттерна

Использование паттерна компоновщик помогает упростить организацию элементов в виде вложенной структуры данных и устраняет избыточность кода при реализации этой задачи. Также компоновщик делает

клиентский объект более простым и позволяет ему обрабатывать элементарные компоненты и их композиции одинаково, в силу их унифицированного интерфейса, определённого классом-компонентом.

Компоновщик упрощает процесс добавления новых компонент. Новые классы, независимо от того, являются они элементарными компонентами или композициями, будут работать с уже существующей на момент их создания структурой. Другими словами нет необходимости изменять клиента при создании новых компонент.

Практический пример использования паттерна

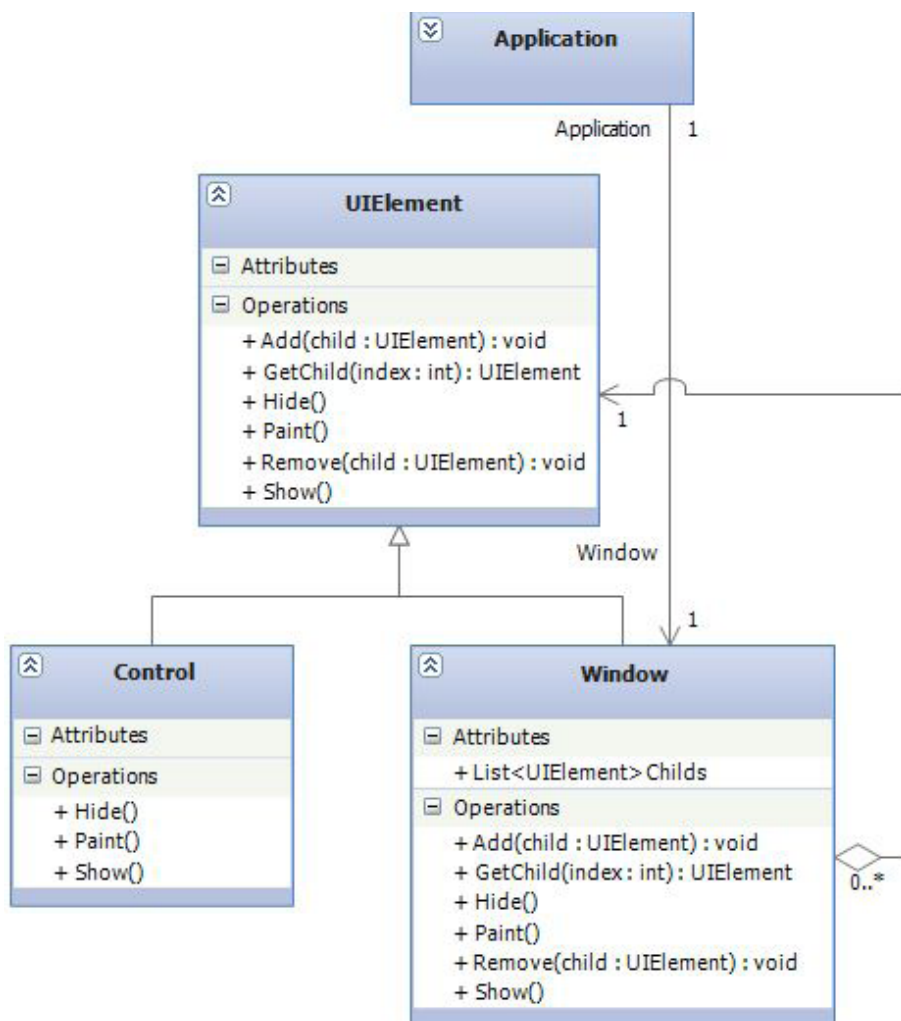
Мы рассмотрим применение паттерна проектирования компоновщик на базе организации множества элементов графического интерфейса пользователя, которое имеет регулярную структуру. Другими словами, всякое окно может содержать элементы управления, но также и другие окна. Окна можно упрощённо определить как композиции элементов управления.

Для организации компоновщика под определённую нами задачу мы объявляем абстрактный класс `UIElement`, описывающий некоторый элемент графического интерфейса пользователя и определяющий интерфейс базового поведения элемента графического интерфейса и интерфейс управления дочерними элементами композиции.

К базовому поведению элемента графического интерфейса относятся методы `Hide`, `Paint` и `Show`, которые соответственно реализуют возможность показывать элемент, перерисовывать его графическое представление и прятать.

Для управления-доступа к дочерним элементам композиции определяются методы Add, GetChild и Remove, которые соответственно позволяют добавлять новый дочерний элемент, получить его и удалить элемент из композиции.

Класс Control, описывает элемент управления, который играет роль «примитивного» элемента реализуемого



нами компоновщика и реализует базовое поведение. Класс `Window` представляет собой композицию элементов управления, но также реализует и базовое поведение, поскольку имеет некоторое элементарное графическое представление.

Класс `Application` играет роль клиента, который ассоциирован с некоторым окном, то есть использует объект класса `Window` и интерфейс, описанный классом `UIElement`.

Модель описанного приложения представлена на диаграмме классов на приведённом выше рисунке.

К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна `Composite`.

5. Паттерн Decorator

Цель паттерна

Цель паттерна *Decorator* (англ. «декоратор») состоит в том, чтобы реализовать возможность динамического добавления функционала к объекту, а так же распределить ответственность за выполнение отдельных функций между отдельными классами.

Так же декоратор представляет собой альтернативу наследованию в смысле расширения функционала объектов.

Причины возникновения паттерна

Достаточно распространённым подходом является разделение ответственности за выполнение отдельных операций между отдельными классами, поскольку это создаёт структуру, при которой каждый класс инкапсулирует логику управления только теми обязанностями, которые на него возложены. И значение имеет не только модульность, упрощающая разработку, но и возможность закрывать классу доступ к операциям, которыми он управлять не должен по его сути.

Одним из методов, позволяющим распределить обязанности по выполнению некоторой общей задачи между отдельными классами является наследование. Однако такой подход является негибким, поскольку при наследовании добавленная функция статически закрепляется для всех потомков этого класса. И для того, чтобы создавать

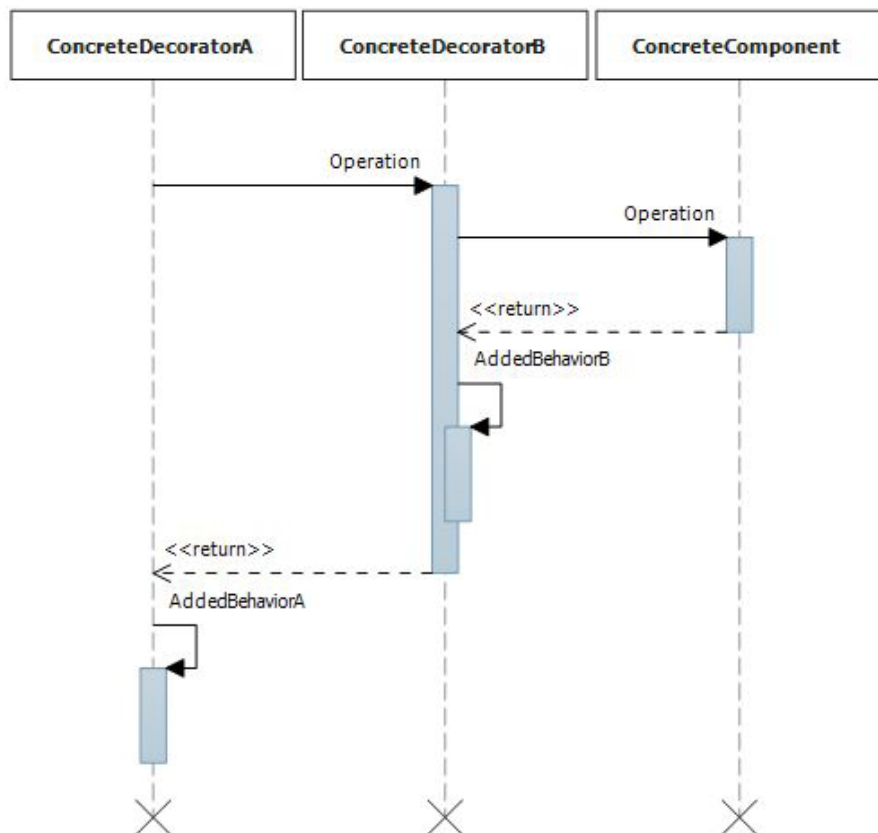
различные типы (с разным набором функциональности), необходимо выполнять наследование «во всех возможных» комбинациях.

Декоратор же позволяет не только гибко комбинировать функциональность объекта по его внутренней логике, но и динамически изменять набор функций объекта во время выполнения, поскольку добавление функции сводится к созданию компонентного объекта необходимого класса.

Структура паттерна

Структура паттерна Decorator представлена следующими элементами:

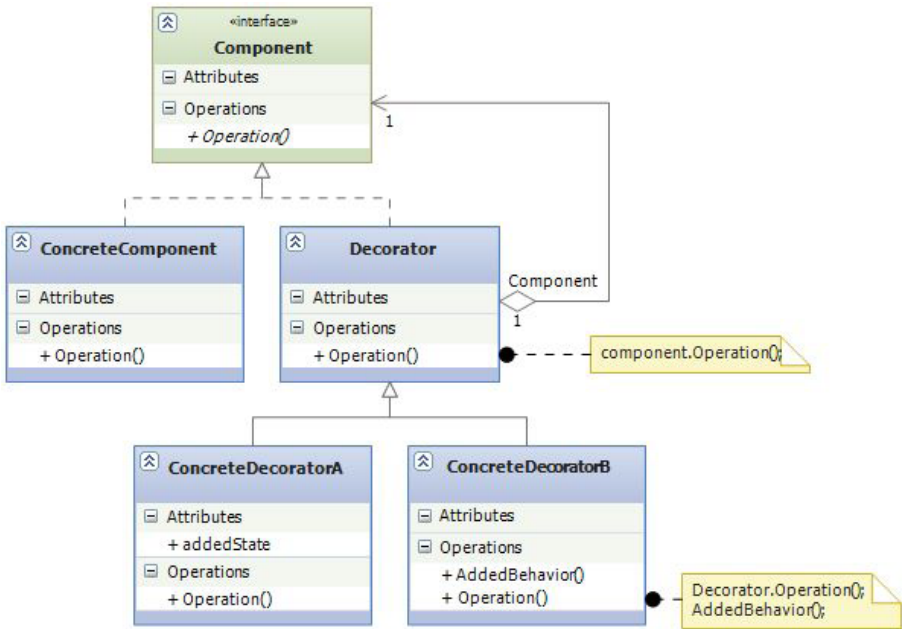
- ***Component*** — представляет собой абстракцию некоторого элемента, для которого будут определены «декорации» (оформление). Конечно же, под «декорированием» понимается не визуальное оформление, а добавление специфических функций. *Component* содержит объявление абстрактной операции, к конкретной реализации которой декоратором, впоследствии, будет добавлен «новый» функционал.
- ***ConcreteComponent*** — представляет собой реализацию компонента.
- ***Decorator*** — класс, который наследует и агрегирует компонент. Он переопределяет реализацию операции таким образом, чтобы выполнить функцию, инкапсулированную в компоненте, а затем добавить новый функционал.



Поскольку декоратор является частным случаем компонента (наследует Component), то в качестве инкапсулированного компонента может быть использован другой декоратор, что позволяет осуществлять рекурсивную последовательность вызовов переопределённой операции с постепенным «накоплением» функционала (как показано выше на диаграмме последовательности).

ConcreteDecoratorA и ConcreteDecoratorB — представляют собой частные реализации декоратора для компонента.

Структура паттерна проектирования декоратор (Decorator) иллюстрируется представленной ниже диаграммой классов.



Результаты использования паттерна

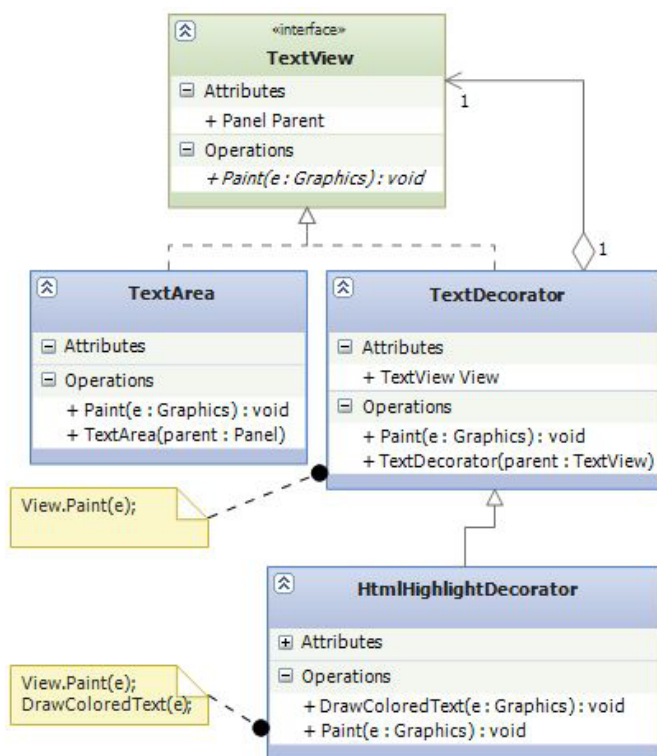
Использование паттерна, как уже выше упоминалось, позволяет более гибко, нежели при использовании наследования реализовать распределение обязанностей по выполнению некоторой сложной задачи между несколькими классами.

С другой стороны паттерн проектирования декоратор предотвращает перенасыщение иерархии классов, поскольку позволяет избежать необходимости создавать классы, которые бы сочетали в себе функционал во всех необходимых комбинациях.

Практический пример использования паттерна

В качестве примера мы рассмотрим модель текстового редактора с подсветкой html-синтаксиса.

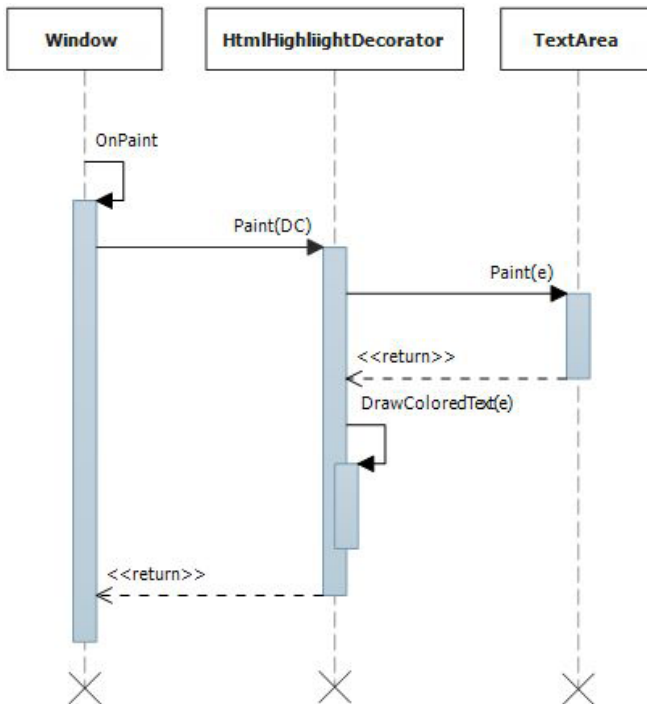
Предполагается использовать класс текстового представления, для реализации логики представления текстовой информации пользователю. Модель реализации текстового представления приведена ниже.



Класс текстового представления (*TextView*) наследуется, с одной стороны, классом конкретной реализации представления в виде текстовой области (класс *TextArea*), поддерживающей редактирование. Для поддержки

возможности расширения функционала текстового представления, нами объявляется класс `TextDecorator`, который, собственно, реализует паттерн `Decorator`. Для реализации возможности подсветки html-синтаксиса нами создаётся конкретная реализация класса `TextDecorator`, которая инкапсулирует логику осуществления подсветки html-синтаксиса в текстовом представлении.

При осуществлении прорисовки окна, в котором используется текстовое представление, (обработчик события `Paint`) создаётся изображение по размеру клиентской области приложения, для которого инициализируется графический контекст. Объект графического контекста созданного изображения передаётся в метод `Paint` объекта класса `HtmlHighlightDecorator`, который вызывает метод `Paint`



объекта класса `TextArea`, а впоследствии и метод `DrawColoredText`, который отвечает за прорисовку подсвеченного текста. Последовательность вызовов иллюстрируется на диаграмме последовательности, приведённой выше.

Таким образом, после возвращения управления в обработчик события `Paint` оконного класса, на созданном в обработчике изображении будет прорисован текст с подсвеченным синтаксисом. Последним действием мы прорисовываем полученное изображение на графическом контексте окна, с которым связано текстовое представление.

```

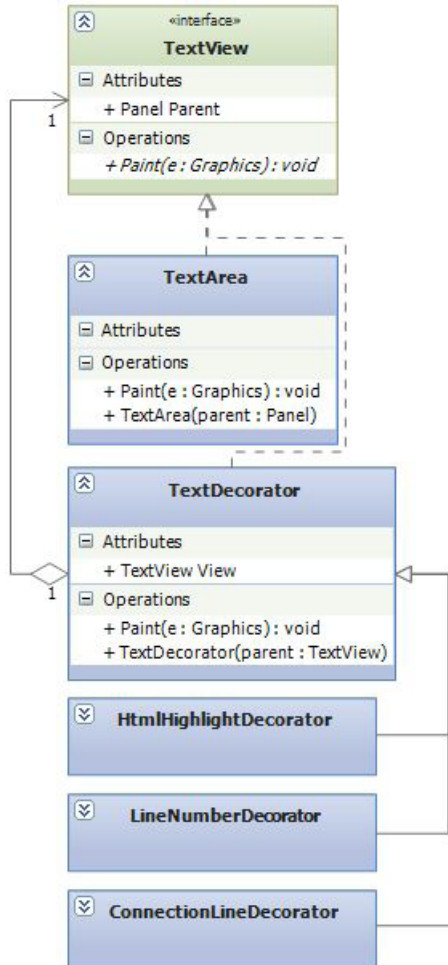
4      <component name=
      "Microsoft-Windows-International-Core-WinPE"
      processorArchitecture="x86" publicKeyToken=
      "31bf3856ad364e35" language="neutral" versionScope=
      "nonSxS" xmlns:wcm=
      "http://schemas.microsoft.com/WMIconfig/2002/State"
      xmlns:xsi=
      "http://www.w3.org/2001/XMLSchema-instance">
5          <SetupUILanguage>
6              <UILanguage>ru-RU</UILanguage>
7          </SetupUILanguage>
8          <InputLocale>en-US; ru-RU</InputLocale>
9          <SystemLocale>ru-RU</SystemLocale>
10         <UILanguage>ru-RU</UILanguage>
11         <UserLocale>ru-RU</UserLocale>
12     </component>

```

Подобная реализация позволит впоследствии гибко добавлять и убирать различные декорирующие эффекты, свойственные текстовым редакторам. Например, номера строк, соединяющие линии для парных элементов, как показано на представленном выше рисунке.

Для этого достаточно объявить несколько декораторов, и добавить их для текстового представления в оконном классе, с которым связано текстовое представление.

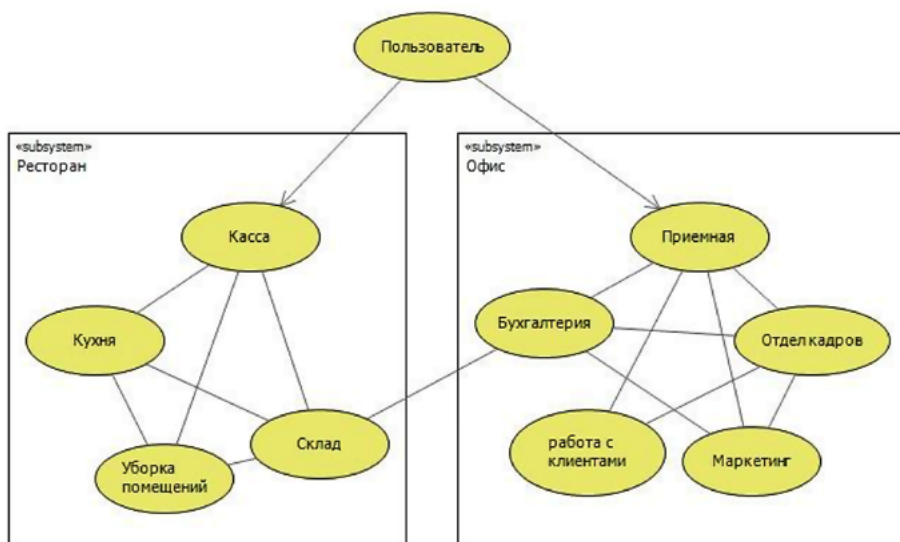
Обобщённая модель приведена ниже.



К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна Decorator.

6. Паттерн Facade

Паттерн **Facade** (*фасад*) предназначен в большей мере для инкапсуляции (сокрытия) содержимого и разделения логических частей на независимые подсистемы. В «реальном» мире много где применяется структура паттерна фасад, хорошим примером служит любой ресторан быстрого питания, где вы просто подходите к кассе и заказываете еду, а дальнейшая структура забегаловки вас, как правило, не интересует. Но в забегаловке также есть офис, в который вы тоже можете прийти, и обратиться к нему по «узкому» интерфейсу (например, стать постоянным клиентом). Таким образом, ресторан получает две минимально зависящие друг от друга подсистемы, которые вместе складываются в одну большую систему.



Цель паттерна

Целью паттерна фасад является определенная структуризация классов, в подсистемы, доступ к каждой из них происходит через один интерфейс. Так же при построении архитектуры приложения желательно его разделять на отдельные под системы, которые должны быть максимально независимы.

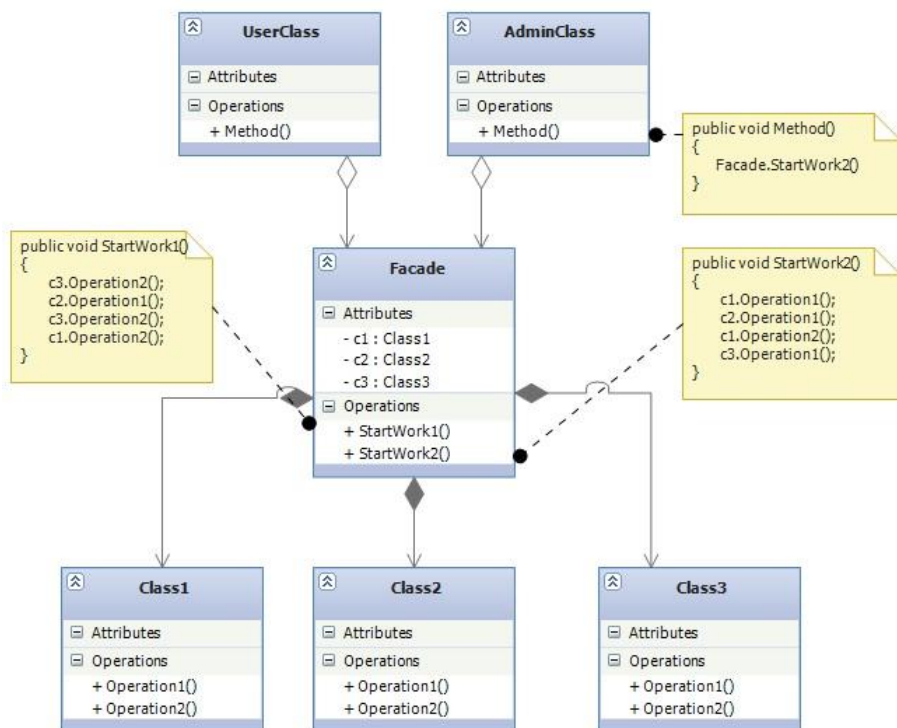
Каждая часть программы (подсистема) будет иметь свой класс фасад, через который будет производиться управление этой частью программы.

Причины возникновения паттерна

Спроектировать и потом построить огромное приложение «сразу» очень сложно, архитекторы решили эту задачу разбиением программы на более мелкие подсистемы, это позволяет строить какую-то одну часть программы не отвлекаясь на другие ее аспекты, после построения переходить к написанию следующей подсистемы. А если строить системы не зависимыми друг от друга можно получить «безопасную» заменимость ее компонентов или целых подсистем, также, получая независимую подсистему, вы можете использовать ее повторно в других проектах, и наконец если в команде программистов более одного человека это позволит поручить каждому(или нескольким) из них написать отдельную подсистему, а потом просто собрать из в целое приложение.

Но для того чтоб воспользоваться незнакомой вам подсистемой приходилось перечитывать немало документации об этой структуре, и разбираться как ней пользоваться. Для решения этой проблемы, был создан паттерн

Фасад(*Facade*). Который подразумевает, что для каждой подсистемы, будет создан свой класс, который является своего рода хранителем подсистемы. Через такой класс будет легко взаимодействовать со структурой.

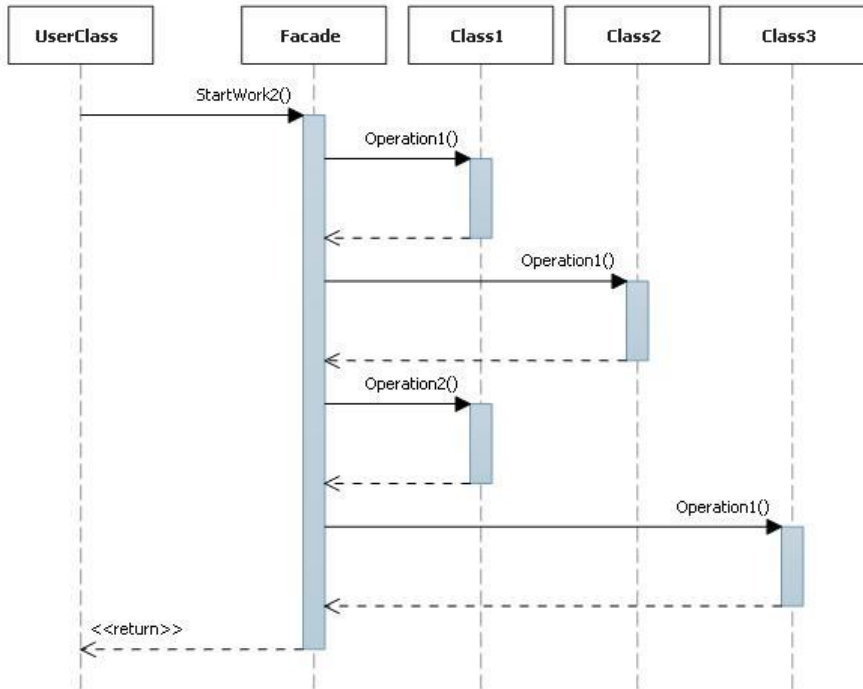


Структура паттерна

Паттерн фасад подразумевает существование некоторой подсистемы в программе, для которой создается класс фасад. Через класс фасада иницируется выполнение операций из классов подсистемы.

Стоит так же отметить то, что клиенты (пользователи подсистемы) не должны иметь доступа к классам подсистемы.

Все объекты подсистемы, если это возможно должны храниться в классе фасад. При вызове клиентами метода из объекта Facade, он начинает работу с классами подсистемы.



Описанный выше пример, не является правилом. В паттерне фасад могут иметься более или менее чем три класса внутри подсистемы. Объект класса фасад может не только вызывать методы из элементов подсистемы, он может их настраивать, присваивать их свойствам значения и так далее.

Необязательно класс фасада должен хранить в себе объекты классов подсистемы.

Доступ к объектам подсистемы можно обустроить через паттерн Proxy. Также иногда программисты делают

класс фасада статическим, для того чтоб к нему можно было обратиться с любой точки программы.

Результаты использования паттерна

Строить программу, начиная с написания отдельных независимых подсистем проще, чем писать сразу всё приложение. Вдобавок, мы получаем готовый блок программы, который можно использовать и в других приложениях. Также можно произвести замену одного из компонентов подсистемы, не нарушая общей структуры приложения.

Если для каждой подсистемы создавать свой класс фасада, пользоваться такой системой будет проще, так как вся работа этой структуры инкапсулирована (спрятана) в классе фасад.

Используя другие паттерны вместе с паттерном Facade можно достичь большей производительности, гибкости, безопасности приложения.

Практический пример

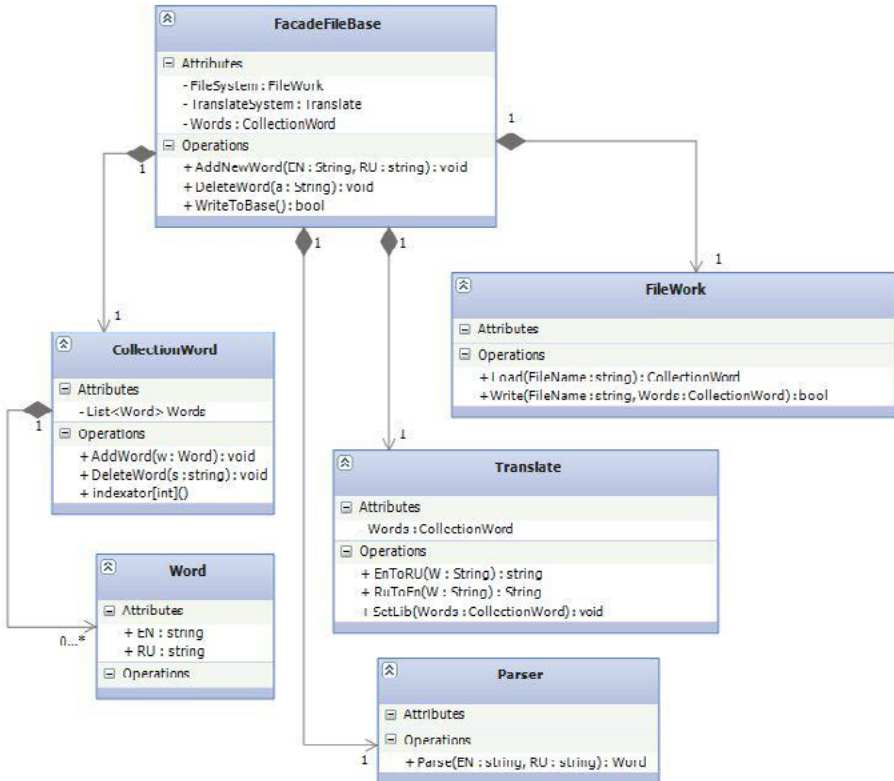
Рассмотрим паттерн фасад в примере следующего приложения: простой переводчик слов, пользователь вводит слово на английском и в результате получает его перевод на русском.

Приложение должно предоставлять возможность работать со словарем заранее записанных слов (сохранять, загружать) с файла. Также приложение должно уметь добавлять новые слова в текстовую базу данных.

В программе предусмотрена подсистема для работы со словарем, доступ к которой будет осуществляться через объект класса фасад.

Далее представлена диаграмма классов этого приложения.

Пользователь будет управлять системой через класс фасад(FacadeFileBase) который в свою очередь вызывает объекты подсистемы.



К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна Facade.

7. Паттерн Flyweight

Цель паттерна

Целью паттерна является более экономное использование памяти компьютера, достигается это более правильным способом работы с большим количеством мелких объектов.

Для понимания паттерна необходимо научиться разделять внутренние и внешние свойства объекта. Внутреннее свойство объекта это свойство, которое хранит объект внутри себя, другими словами это экземпляр класса внутри которого создано свойство (переменная) и храниться до тех пор, пока «живет» объект. Внешнее свойство это свойство, которое передается объекту как аргумент одного (или нескольких) метода, и существует, пока выполняется метод.

Суть паттерна заключается в том чтобы «переписать» внутренние свойства во внешние, и после не создавать много объектов, а создать один который будет «отображаться» по разному, в зависимости от того какие внешние свойства к нему будут применены.

Причины возникновения паттерна

После «перехода» на ООП стало проще программировать так как все объекты «как настоящие», но это сопряжено с некоторым количеством трудностей. Создавая модель приложения, хочется расположить объекты по принципам объектно-ориентированного программирования, например стул состоит из ножек и сидения, а сидение состоит

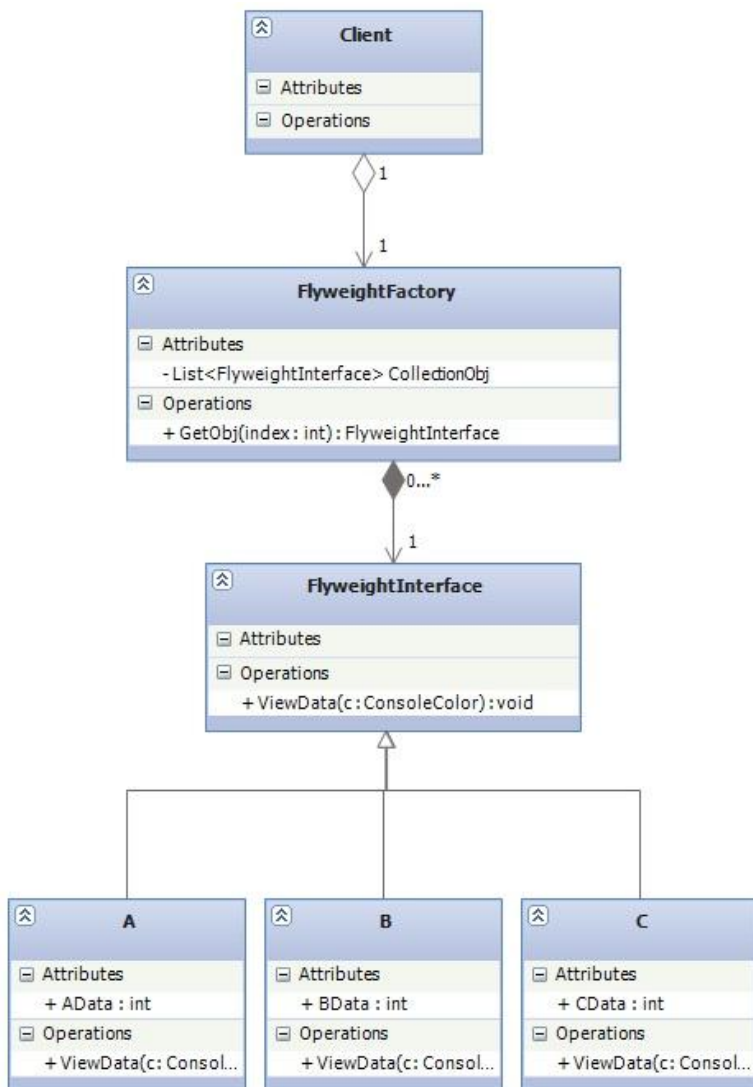
еще из нескольких деталей. Если описывать такую архитектуру слишком «глубоко» может возникнуть ситуация когда объектов будет слишком много, от чего приложение с такой архитектурой будет «тормозить». Что же делать когда необходимый уровень «глубины» недостижим для компьютера? Ответ: разделить логическое понимание приложение от физического, вить в отличии от реальной жизни в программировании можно встретить моменты когда одна и та же сущность находится «рисуеться» в двух (или более) местах. Так например Логически у нас есть стул, состоящий из ножек и сидения, а физически в программе создан объект ножки которая фигурирует в разных местах. Для того чтобы объект фигурировал в разных местах достаточно просто при вызове из объекта метода «отобразить» передавать в него, некоторое количество параметров которые и будут указывать где и как отображать этот элемент. Используя такой подход к построению архитектуры программист получит приложение которое будет достаточно экономно использовать ресурсы компьютера.

Структура паттерна

Объект который «фигурирует» в нескольких местах одновременно называется приспособленцем. Доступ к таким объектам предоставляется посредством фабрики. Клиент создает объекты приспособленцев с помощью фабрики, вызывая метод, который возвращает объект приспособленца и принимает идентификатор объекта, который ему необходимо вернуть.

Фабрика хранит в себе (коллекцию, массив, отдельно) объектов приспособленцев. Важно что объекты должны

создаваться при первом запросе клиента, а при последующих запросах им должен возвращаться уже созданный объект который как было сказано выше должен храниться внутри фабрики.



При вызове метода `GetObj` пользователь передает в него идентификатор объекта, который ему необходимо получить, если объект с таким индексом уже создан, его необходимо вернуть как результат работы метода, но если объект не создан, его необходимо создать, и записать в хранилище фабрики, после чего вернуть как его как результат работы метода.

Классы `A`, `B`, `C` наследуются от базового класса, который описывает интерфейс взаимодействия со всеми типами приспособленцев.

Результаты использования паттерна

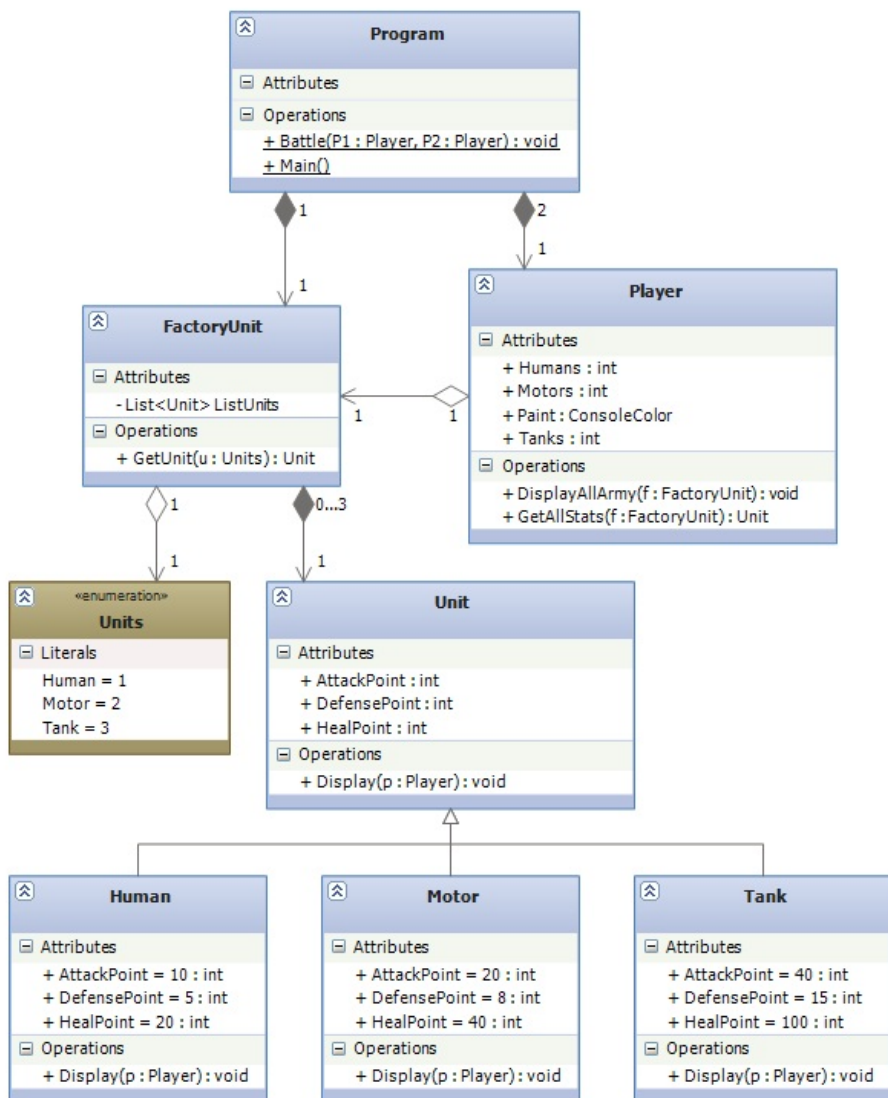
Если применять этот паттерн к своему приложению, вы получите действительно сильную экономию памяти, даже учитывая то, что при получении доступа к объекту будет теряться время в методе фабрики (в нашем случае `GetObj`). Но экономия памяти получится лишь в том случае, если в объектах по минимуму будут содержаться внутренние свойства, а при вызове будут передаваться внешние. Внешние свойства должны вычисляться алгоритмами, в этом и состоит основная идея паттерна, «экономить память за счет вычислительной мощности».

Получать доступ к объектам приспособленцев стоит только через методы фабрики, так как это добавит гибкости и возможности настроить объект в методах фабрики.

Применять паттерн следует тогда когда в вашем приложении действительно большое количество объектов, и когда их состояние можно описать внешними свойствами, и внешние свойства можно описать и присвоить с помощью вычислений.

Практический пример

Рассмотрим консольное приложение, которое будет отображать, и просчитывать итог сражения двух армий.



Классы Human, Motor и Tank, отличаются друг от друга только значениями внутренних свойств, и методом, отображающим на экране боевую единицу.

Объект класса FactoryUnit будет хранить в себе от нуля до трех объектов в виде Unit в специально для этого созданной коллекции ListUnits. Классы клиентов смогут получить доступ, к объекту вызвав из фабрики метод и передав в него объект перечисления который и укажет какой конкретно Unit надо вернуть как результат вызова метода.

Класс игрока(Player) будет использовать фабрику для получения и отображения свойств объектов. Но пользоваться все объекты игроков будут на самом деле только 3-мя экземплярами классов потомков Unit это и должно экономить память.

Основной класс Program содержит метод Main, в котором происходит управление всей программой, а так же метод Battle отвечающий за математику которая покажет какой был исход битвы армий двух игроков.

Пример кода, который демонстрирует работу паттерна Flyweight прикреплен к pdf-файлу данного урока.

8. Паттерн Proxy

Паттерн Proxy часто именую как паттерн суррогат. Далее основной класс, о котором будет идти речь, именуется суррогатом, или прокси, оба термина являются верными и описывают один и тот же класс / объект.

Прокси это — некоторый объект, обеспечивающий транзитный доступ к другому объекту.

Суррогат это — некоторый объект, который внешне ничем не отличается от основного, но внутренней функциональности в нем нет.

Объект Суррогата или прокси внешне ничем не отличается от объекта, который он представляет, но вся его внутренняя функциональность лишь является некоторым туннелем, через который классы клиента получают доступ к основным объектам.

Так же объект суррогата или прокси можно называть заместителем объекта.

Цель паттерна

Целью паттерна является создание системы доступа к объекту через специальный объект суррогата.



Это позволяет добиться большей гибкости работы с объектом целевого класса.

Если объект целевого класса после создания будет занимать много места и нет гарантии того что объект будет необходим в работе приложения, можно создавать целевой объект только когда клиент первый раз вызывает метод из объекта суррогата это ускорит быстроедействие приложения в случае если объект не будет использоваться. А если объект и будет создан, то на этапе выполнения программы это будет не так «болезненно» для пользователя.

Такая система доступа позволит так же использовать целевой класс с большей защитой (в суррогате можно выполнять различные проверки принимаемых параметров). Например, если целевой класс это калькулятор, просто принимает в аргументы метода два параметра и выполняет базовые математические операции, в суррогате класса калькулятор можно выполнить проверку чтобы не получилось, что калькулятору будет передана задача, «поделить на ноль».

Можно, использовать структуру паттерна прокси для создания так называемого «Посла» в другое пространство имен. Например, целевой класс описан в пространстве имен, которое по каким-то причинам не возможно (или неудобно) подключить, в таком случае можно создать суррогат, который будет находиться в «нужном» пространстве имен, а внутри суррогата будет создан объект целевого класса из другого пространства имен.

Причины возникновения паттерна

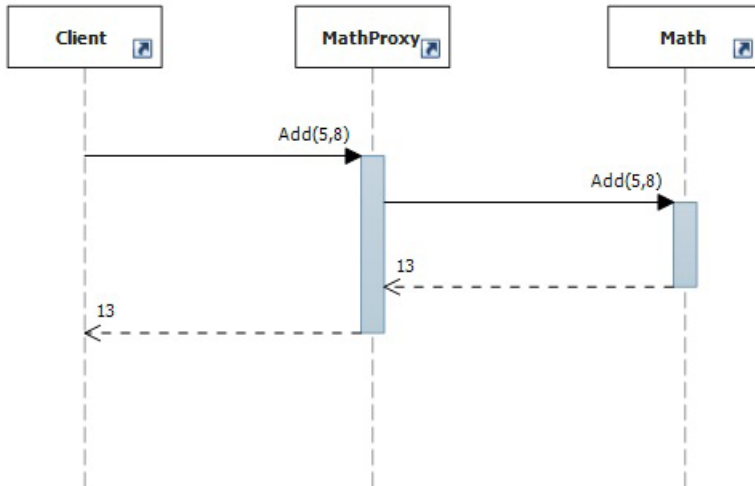
Код одних программистов должен быть понятен другим программистам, касательно как синтаксических «традиций» так и структурных. Если строить приложение,

используя паттерн прокси для доступа к объектам и в объектах суррогатов заниматься проверкой данных, а в самом целевом классе выполнять логику. Потом достаточно легко объяснить другому программисту, где выполняются проверки, а где основная логика. Вообще используя паттерны довольно легко объяснить структуру приложения.

Наверное, вам уже приходилось видеть приложения, которые после начала запуска, очень долго «думают» и только по истечении 5 или более секунд начинают отображать свой интерфейс. Так получается, потому что приложение сразу при старте начинает загружать все свои модули, создает все экземпляры классов, которые понадобятся (или не понадобятся) в работе приложения. Для частичного решения такой проблемы, можно создать суррогат «тяжелого» объекта. Создавать такой объект при старте приложения, суррогат это лишь муляж который не является таким «тяжелым» как целевой объект и потому создается быстрее, а «тяжелый» объект будет создан только, тогда когда он будет действительно необходим. А необходимость появляется, когда классы клиента будут работать с объектом суррогата, который все запросы будет перенаправлять на только что созданный целевой объект.

Структура паттерна

Паттерн Проху подразумевает наличие некоторого объекта(далее «целевой класс»), к которому будет осуществляться доступ с помощью специально созданного объекта суррогата.



Но объект суррогата не должен отличаться внешне от объекта целевого класса, потому что оба класса должны быть унаследованы от одного интерфейса.

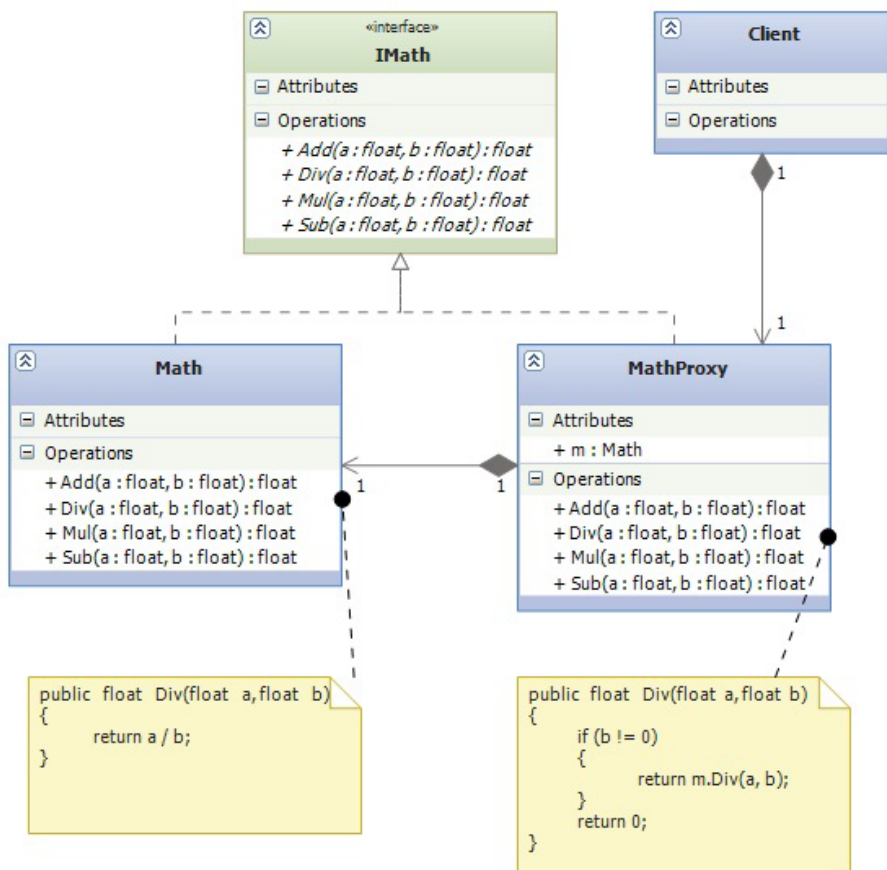
В приведённом далее примере целевой объект `Math` создается сразу с объектом суррогата, так как класс `math` не является «тяжелым» и будет создаваться быстро.

Пример показывает, как с помощью объекта суррогата можно выполнять различные проверки перед непосредственной передачей данных целевому классу.

Результаты использования паттерна

Реализовав доступ к паттерну через объект суррогата, мы получаем дополнительный уровень работы с целевым объектом. На этом уровне мы можем заниматься оптимизацией, создавая целевые объекты только по мере того как они будут использоваться. Можно заниматься проверкой данных перед передачей в целевой объект. В объекте суррогата можно даже вести логирование

доступа к объекту (например, высчитывать сколько раз был вызван метод суммирования на калькуляторе).



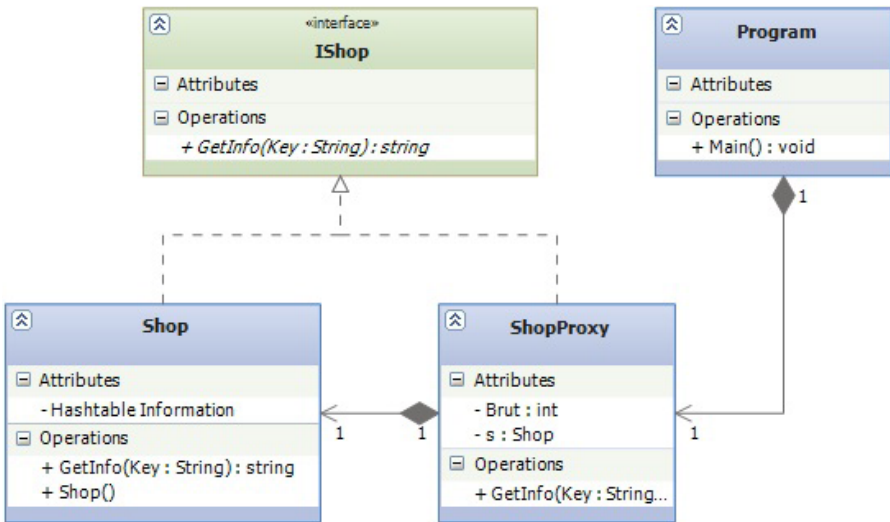
Практический пример

Для лучшего понимания паттерна ознакомимся со структурой приложения «магазин информации». В архитектуре приложения имеется класс `Shop` к которому осуществляется доступ через класс `ProxyShop`. Класс `Shop`

имеет метод который принимает строку с ключом, а потом если возможно возвращает строку с информацией.

Класс ProxyShop защищает класс Shop от возможности его «брутфорсить» также объект Shop создается только тогда когда он будет необходим в программе, другими словами тогда когда кто-то попытается получить информацию по ключу.

Ниже представлена диаграмма классов этого приложения.



К pdf-файлу данного урока прикреплен архив где вы можете найти реализацию паттерна Proxy.

9. Анализ и сравнение структурных паттернов

Вы уже изучили все из структурных паттернов:

- **Adapter** — паттерн который позволяет «адаптировать» объект под другой интерфейс, для доступа к нему. Например в объекте имеется метод `Operation1` а нам необходимо сделать так чтоб он назывался `OperationA`. Целью паттерна `Adapter` является «редактирование» интерфейса доступа к целевому объекту.
- **Bridge** — паттерн позволяет отделить абстракцию от реализации, когда имеется иерархия объектов которая описана абстрактно и позже будет реализоваться иерархия под конкретную систему. Целью паттерна `Bridge` является построение отдельно абстракции и реализации и предоставление клиенту абстракции с помощью, которой он сможет управлять реализацией.
- **Composite** — структурный паттерн который выстраивает объекты по типу дерева. Целью этого паттерна является построение древовидной структуры для хранения объектов.
- **Decorator** — паттерн позволяющий структурировать таким образом что несколько объектов отображаются как один. И количество внутренних объектов может изменяться «на лету».
- **Facade** — структурный паттерн который рассказывает как строить большие приложения из мелких не

зависимых подсистем. Целью паттерна является создание узкого и понятного интерфейса для работы с подсистемой.

- ***Flyweight*** — паттерн позволяющий экономить место в случае если в программе имеется множество мелких объектов и их состояние можно вынести во внешние свойства которые вычисляются алгоритмами.
- ***Proxy*** — подразумевает создание объекта суррогата для целевого объекта чем может обеспечить большую гибкость, экономию памяти, защиту.

Подходить к использованию паттернов нужно отталкиваясь от их назначения, потому что с первого взгляда может показаться что паттерны многим похожи, но так кажется только потому что у некоторых из них похожая структура. Но у каждого паттерна есть цель использования. Например, вам может показаться паттерны фасад и прокси, очень похожи, но предназначены они для разных целей, цель паттерна фасад, создать узкий интерфейс работы с подсистемой, а прокси позволяет защитить целевой объект.

Структурно паттерны Composite и Decorator тоже очень похожи, но необходимо отталкиваться от целей применения, так как паттерн Composite структурирует объекты, а паттерн Decorator позволяет создать отдельные виды функциональности и создать из них «один» объект.

Часто приходится видеть, как люди не видят разницы между паттерном фасад(*Facade*) и паттерном адаптер(*Adapter*) потому что кажется, что цель у них одна и так же, это изменить или модифицировать интерфейс доступа к объекту или объектам. Но адаптер позволяет

работать со старым интерфейсом доступа к объекту и использовать новый, а паттерн Façade создает новый интерфейс, более целенаправленный.

Важно отметить тот факт что паттерн это не эталон структуры программы, на практике приходится слышать, «ну вот у нас здесь архитектура как в паттерне Flyweight но в фабрике разные типы объектов возвращаются из разных методов».

10. Домашнее задание

В качестве домашнего задания необходимо разработать три модели приложений, каждое из которых должно реализовывать хотя бы один структурный паттерн проектирования, описанный в текущем уроке. Каждая модель должна быть представлена в виде одной диаграммы классов, отражающей структуру классов, используемых этой моделью, а также связи, в которых состоят эти типы данных.

Одну из созданных моделей (на выбор) необходимо реализовать в виде работающего приложения.

Компьютерная Академия ШАГ www.itstep.org

Все права на охраняемые авторским правом фото-, аудио- и видеопроизведения, фрагменты которых использованы в материале, принадлежат их законным владельцам. Фрагменты произведений используются в иллюстративных целях в объеме, оправданном поставленной задачей, в рамках учебного процесса и в учебных целях,

соответствии со ст. 21 и 23 Закона Украины «Про авторське право і суміжні права». Объем и способ цитируемых произведений соответствует принятым нормам, не наносит ущерба нормальному использованию объектов авторского права и не ущемляет законные интересы автора и правообладателей. Цитируемые фрагменты произведений на момент использования не могут быть заменены альтернативными, не охраняемыми авторским правом аналогами, и как таковые соответствуют критериям добросовестного использования и честного использования.

Все права защищены. Полное или частичное копирование материалов запрещено. Согласование использования произведений или их фрагментов производится авторами и правообладателями. Согласованное использование материалов возможно только при указании источника.

Ответственность за несанкционированное копирование и коммерческое использование материалов определяется действующим законодательством Украины.